

江西师范大学数字产业学院学生实验报告

专业： 数据科学与大数据技术 姓名： 王玮霖 学号： 202329003004 日期： 2024.10.30

课程名称	数据结构（实践）	实验室名称	计算机综合实验室
实验名称	排序		
指导教师	陈志威	成绩	

1. 实验目的

- （1）熟练掌握图的邻接矩阵与邻接表存储结构及其应用方法。
- （2）掌握掌握深度优先及广度优先算法，能利用两种算法求解相关问题。
- （3）理解并掌握最小生成树算法的基本思想及其实现方法
- （4）理解并掌握最短路径算法的基本思想及其实现方法。
- （5）理解并掌握拓扑排序算法的基本思想及其实现方法。

2. 实验内容

- （1）请设计直接插入排序算法函数 `void insertSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。。
- （2）请设计二分插入排序算法函数 `void binInsertSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。
- （3）请设计 `shell` 排序算法函数 `void shellSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。
- （4）请设计简单选择排序算法函数 `void selectSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。
- （5）请设计筛选函数 `void sift(int a[],int k,int n)`，对 `a[k]` 进行筛选，并利用其设计堆排序算法函数 `void heapSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。
- （6）请设计冒泡排序算法函数 `void bubbleSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。
- （7）请设计快速排序算法函数 `void quickSort(int a[],int low,int right)`，对 `a[low]..a[right]`进行升序排序。并测试在不同数据规模下的排序效率。
- （8）请设计归并排序算法函数 `void mergeSort(int a[],int n)`，对 `a[1]..a[n]`进行升序排序。并测试在不同数据规模下的排序效率。
- （9）请设计基于链表的基数排序函数 `void radixSort(linklist head)`，对带头结点的整型非负单链表进行升序排序。并测试在不同数据规模下的排序效率。

3. 算法描述及实验步骤

(1) 取出未排序序列中的第一个元素，在已排序序列中从后向前扫描，找到插入位置同时大于插入元素的元素后移，插入元素。

(2) 取出未排序序列中的第一个元素，在已排序序列中使用二分查找确定插入位置，大于插入元素的元素后移，插入元素。

(3) 初始间隔为 $n/2$ ，逐步缩小间隔，对每个子序列进行插入排序，取出未排序序列中的元素，在子序列中从后向前扫描，找到插入位置，元素后移，插入元素。

(4) 假设当前未排序部分的第一个元素是最小的，在未排序部分中找到最小元素的下标，将最小元素与未排序部分的第一个元素交换。

(5) sift 函数，首先，将 $a[k]$ 的值保存在 $a[0]$ 中，作为临时变量。然后，从 k 开始，向下调整堆结构。对于当前节点 i ，找到其左右子节点 $2*i$ 和 $2*i+1$ 。如果右子节点存在且比左子节点大，则选择右子节点作为较大的子节点。如果当前节点的值大于或等于较大的子节点的值，则调整结束，否则将较大的子节点的值上移到当前节点，并继续向下调整。最后，将最初保存在 $a[0]$ 中的值放到最终调整到的位置；heapSort 函数，从数组的中间位置开始，向前遍历数组，对每个元素调用 sift 函数，将堆顶元素与堆的最后一个元素交换，然后将堆的大小减一，并对新的堆顶元素调用 sift 函数，重新调整堆结构。重复这个过程，直到堆中只剩下一个元素，此时数组已经按升序排列。

(6) 内层循环将较大的元素“冒泡”到末尾，交换相邻元素，标记发生了交换，结束循环，如果未发生交换，说明数组已经有序，提前退出。

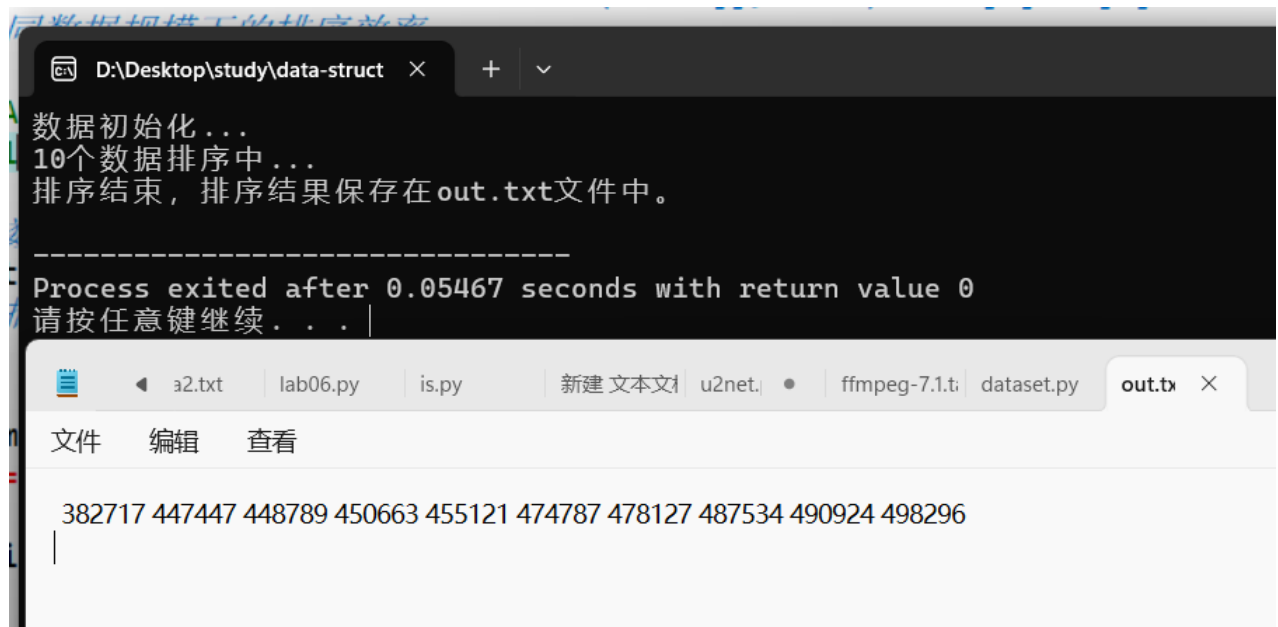
(7) 从数组中选择一个元素作为基准，将数组重新排列，使得所有小于等于基准元素的元素都位于基准的左侧，所有大于基准元素的元素都位于基准的右侧。分区完成后，基准元素位于其最终排序后的正确位置。对基准左侧的子数组递归调用快速排序。对基准右侧的子数组递归调用快速排序。当子数组的长度为 0 或 1 时，递归终止（因为长度为 1 的数组已经是有序的）。

(8) merge 函数，将数据复制到临时数组，合并临时数组到原数组，复制剩余元素；mergepass 函数，对 $a[1..n]$ 进行长度为 len 的一趟并归，处理剩余部分，返回下一趟的归并长度；mergeSortdc 函数，计算中间位置，递归排序左半部分，递归排序右半部分，合并两部分。

(9) 链表为空或只有一个节点，无需排序，初始化队列，找到链表中的最大值，确定排序的位数，按每一位进行分配和收集，链表末尾置空。

4. 调试过程及运行结果

(1) 实验 lab10_01:



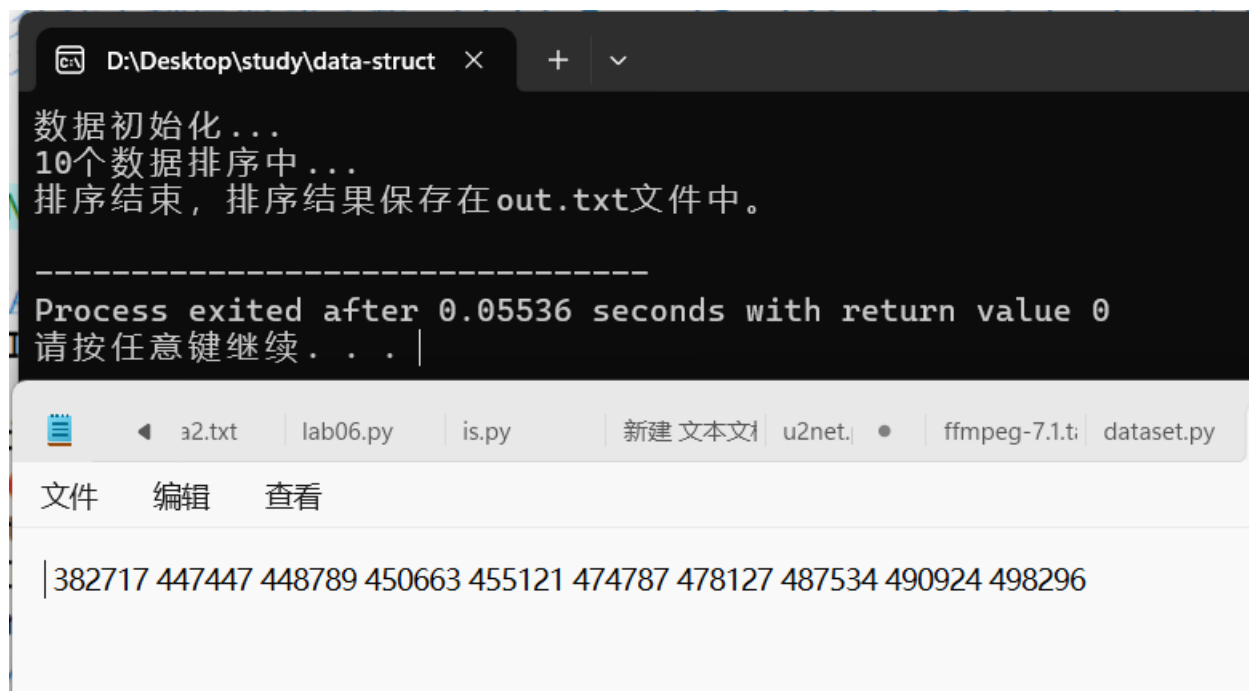
```
D:\Desktop\study\data-struct > .\lab06.py
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.05467 seconds with return value 0
请按任意键继续. . .
```

文件 编辑 查看

382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

(2) 实验 lab10_02:



```
D:\Desktop\study\data-struct > .\lab06.py
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.05536 seconds with return value 0
请按任意键继续. . .
```

文件 编辑 查看

382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

(3) 实验 lab10_03:



The screenshot shows a Windows command prompt window with the following text:

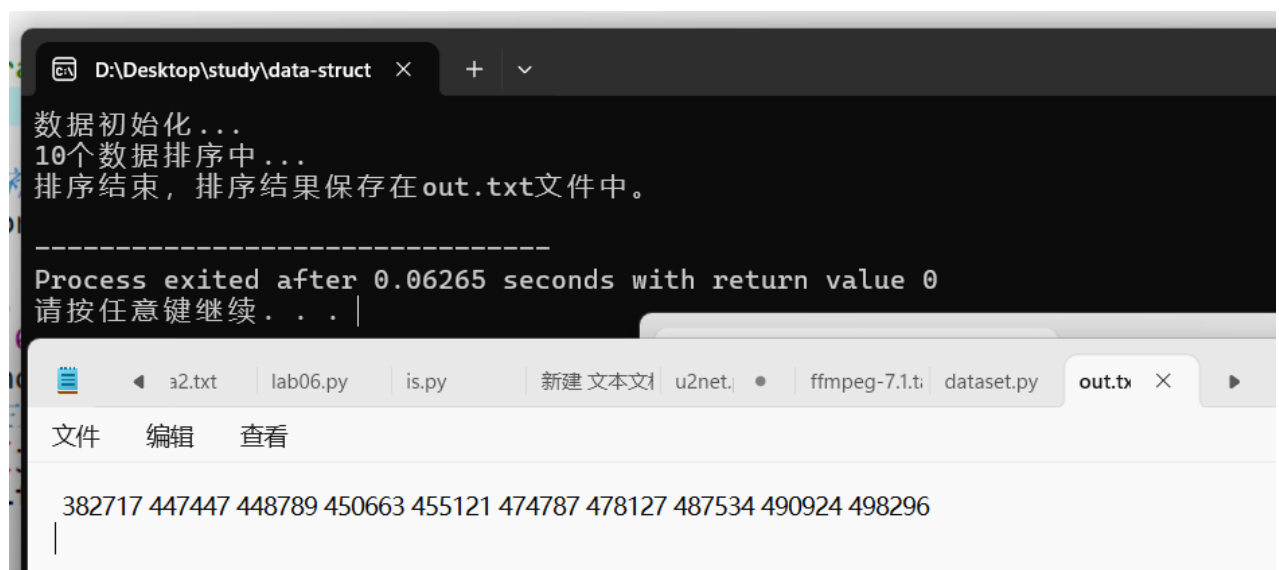
```
D:\Desktop\study\data-struct >
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.05031 seconds with return value 0
请按任意键继续. . .
```

Below the command prompt is a text editor window with the following tabs: a2.txt, lab06.py, is.py, 新建 文本文档, u2net.py, ffmpeg-7.1.t, dataset.py. The text editor shows the following text:

```
文件 编辑 查看
382717 447447 448789 450663 455121 474787 478127 487534 490924 498296
|
```

(4) 实验 lab10_04:



The screenshot shows a Windows command prompt window with the following text:

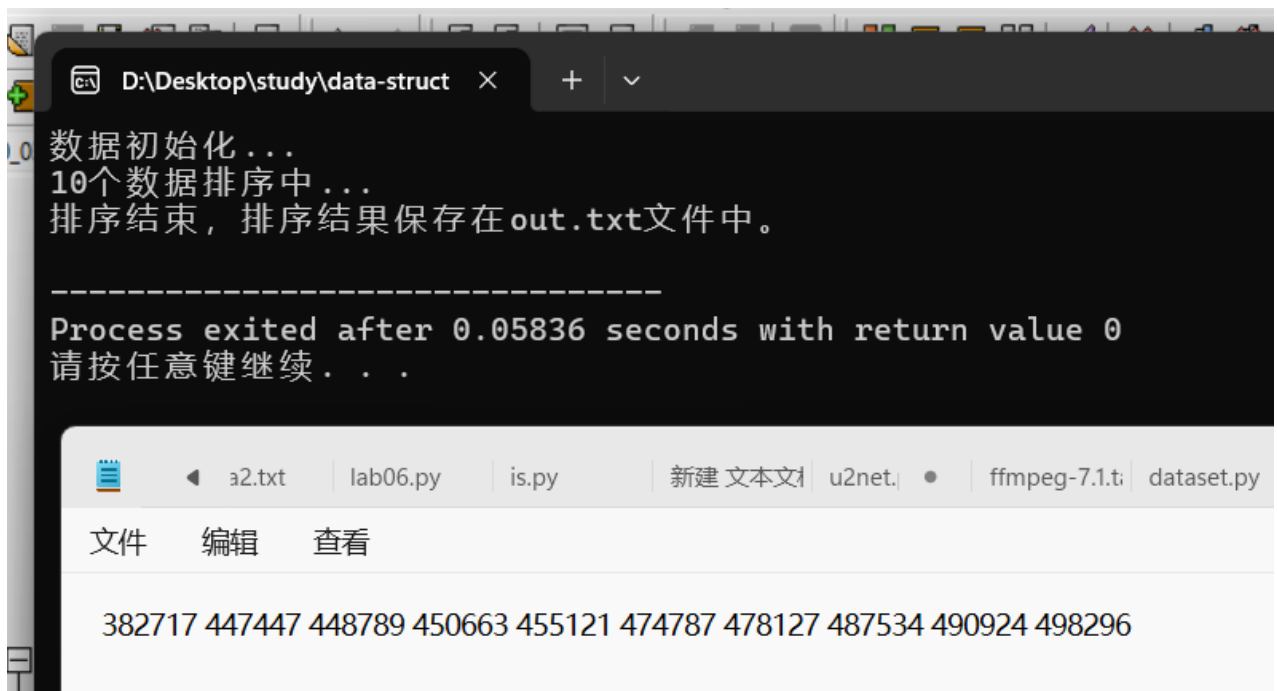
```
D:\Desktop\study\data-struct >
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.06265 seconds with return value 0
请按任意键继续. . .
```

Below the command prompt is a text editor window with the following tabs: a2.txt, lab06.py, is.py, 新建 文本文档, u2net.py, ffmpeg-7.1.t, dataset.py, out.txt. The text editor shows the following text:

```
文件 编辑 查看
382717 447447 448789 450663 455121 474787 478127 487534 490924 498296
|
```

(5) 实验 lab10_05:



```
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.05836 seconds with return value 0
请按任意键继续. . .
```

382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

(6) 实验 lab10_06:

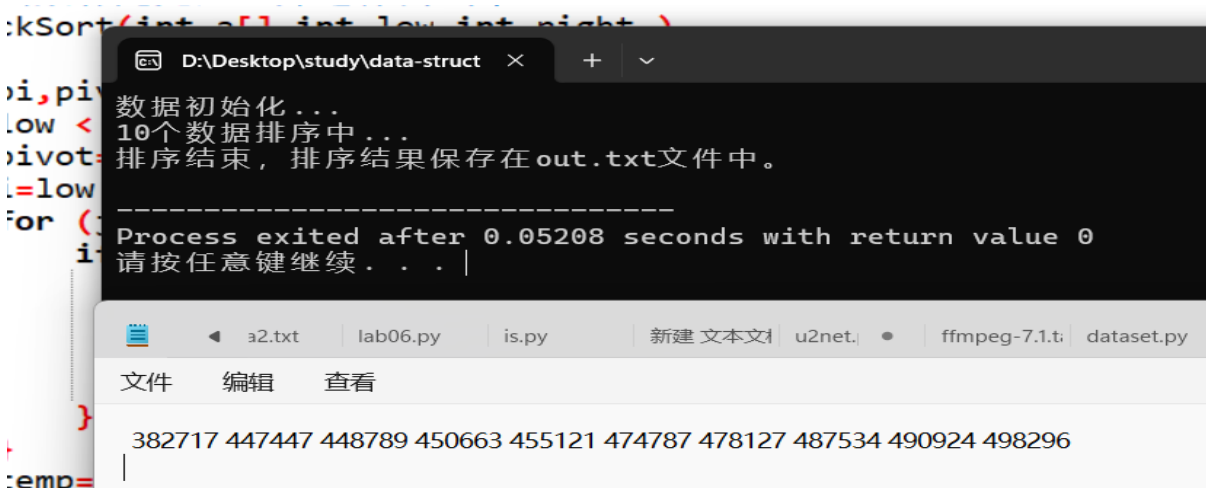


```
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.05208 seconds with return value 0
请按任意键继续. . .
```

382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

(7) 实验 lab10_07:

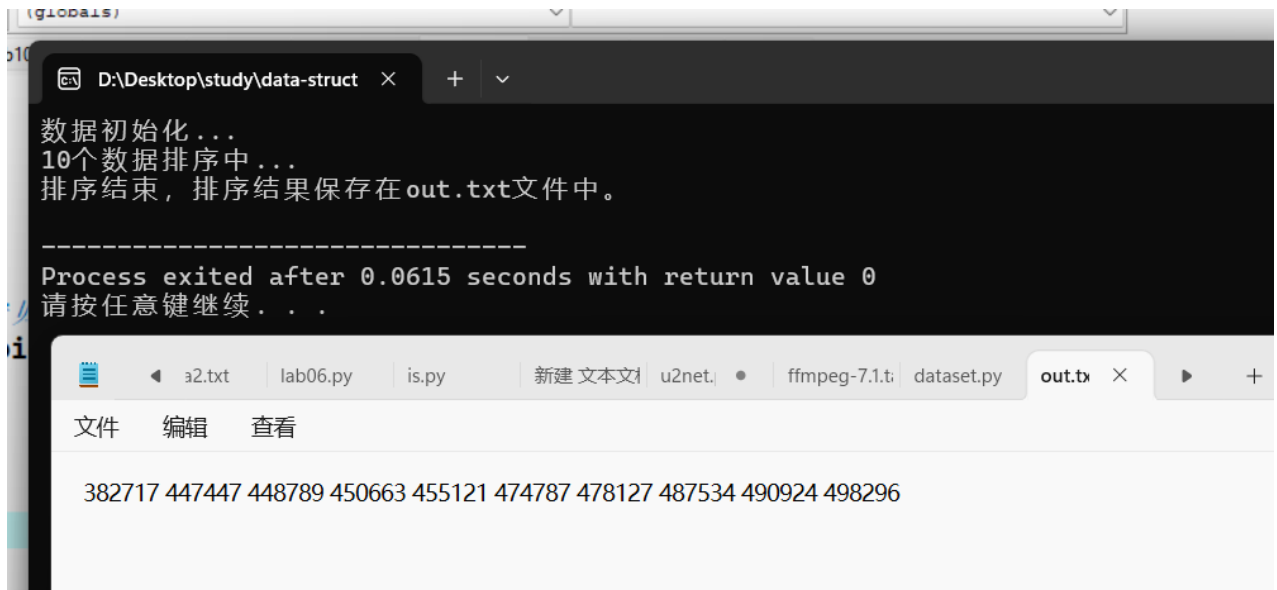


```
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.05208 seconds with return value 0
请按任意键继续. . .
```

382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

(8) 实验 lab10_08:

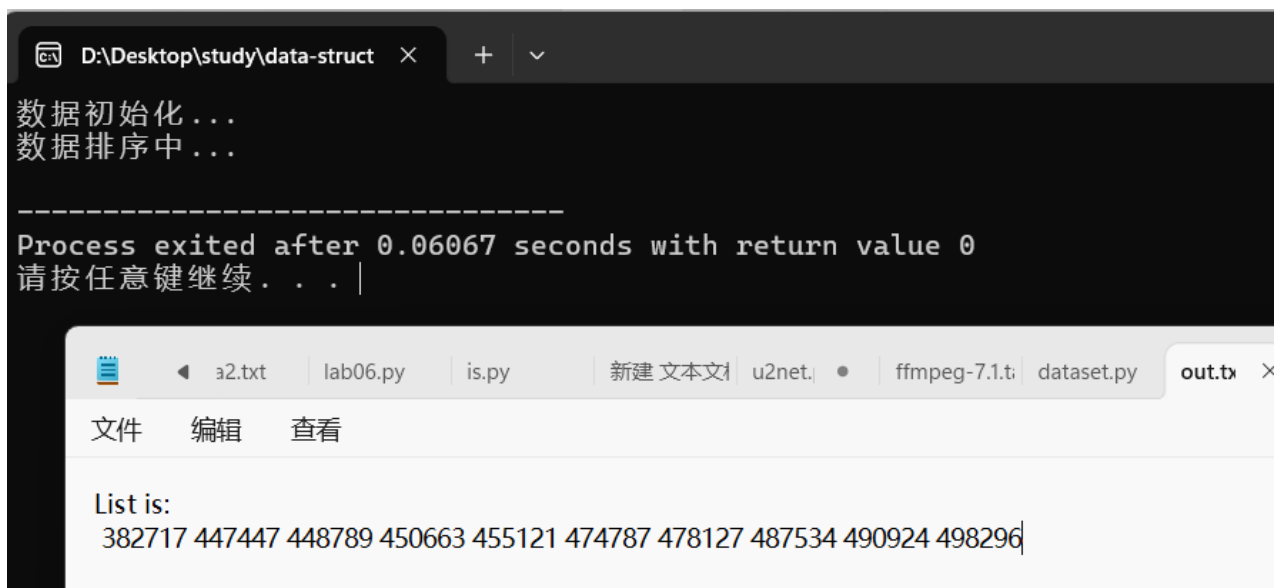


```
数据初始化...
10个数据排序中...
排序结束, 排序结果保存在out.txt文件中。

-----
Process exited after 0.0615 seconds with return value 0
请按任意键继续. . .
```

382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

(9) 实验 lab10_09:



```
数据初始化...
数据排序中...

-----
Process exited after 0.06067 seconds with return value 0
请按任意键继续. . . |
```

List is:
382717 447447 448789 450663 455121 474787 478127 487534 490924 498296

5. 总结

对于排序有了更加清晰的了解, 学习到了不少有关算法。

6. 附录

Lab1:

```
1  /*
2  请设计直接插入排序算法函数 void insertSort(int a[],int n)，对 a[1]..a[n]进行升序
   排序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值时
   需让 N<=500000*/
7
8  /*请将本函数补充完整，并进行测试*/
9  void insertSort(int a[],int n)
10 {    /*直接插入排序*/
11     int i, j, temp;
12     for (i = 1; i < n; i++) {
13         temp = a[i];    // 取出未排序序列中的第一个元素
14         j = i - 1;
15         // 在已排序序列中从后向前扫描，找到插入位置
16         while (j >= 0 && a[j] > temp) {
17             a[j + 1] = a[j];    // 元素后移
18             j--;
19         }
20         a[j + 1] = temp;    // 插入元素
21     }
22 }
23
24 int main()
25 {
26     int a[N+1],n;                /*数据存储在 a[1]...a[N] 中*/
27     printf("数据初始化...\n");
28     n=readData(a,N,"data1.txt");    /*从 data1.txt 中读入 N 个整数存入数组 a, n
   为实际读入的数据个数*/
29     printf("%d 个数据排序中...\n",n);
30     insertSort(a,n);
31     saveData(a,n,"out.txt");        /*排序结果存放在 out.txt 文件中*/
32     printf("排序结束，排序结果保存在 out.txt 文件中。\\n");
33     return 0;
34 }
```

Lab2:

```
1  /*
2  请设计二分插入排序算法函数 void binInsertSort(int a[],int n)，对 a[1]..a[n]进行
   升序排序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值
   时需让 N<=500000*/
7
8  /*请将本函数补充完整，并进行测试*/
9  void binInsertSort(int a[],int n)
10 {
11     int i, j, left, right, mid, temp;
12     for (i = 1; i < n; i++) {
13         temp = a[i]; // 取出未排序序列中的第一个元素
14         left = 0;
15         right = i - 1;
16         // 使用二分查找确定插入位置
17         while (left <= right) {
18             mid = (left + right) / 2;
19             if (a[mid] > temp) {
20                 right = mid - 1;
21             } else {
22                 left = mid + 1;
23             }
24         }
25         // 将元素插入到正确位置
26         for (j = i - 1; j >= left; j--) {
27             a[j + 1] = a[j]; // 元素后移
28         }
29         a[left] = temp; // 插入元素
30     }
31 }
32
33 int main()
34 {
35     int a[N+1],n;                /*数据存储在 a[1]...a[N] 中*/
36     printf("数据初始化...\n");
37     n=readData(a,N,"data1.txt"); /*从 data1.txt 中读入 N 个整数存入数组 a, n
   为实际读入的数据个数*/
38     printf("%d 个数据排序中...\n",n);
39     binInsertSort(a,n);
40     saveData(a,n,"out.txt");    /*排序结果存放在 out.txt 文件中*/
```

```
41.     printf("排序结束，排序结果保存在 out.txt 文件中。\\n");
42.     return 0;
43. }
```

Lab3:

```
1  /*
2  请设计 shell 排序算法函数 void shellSort(int a[],int n)，对 a[1]..a[n]进行升序排
   序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值
   时需让 N<=500000*/
7
8  /*请将本函数补充完整，并进行测试*/
9  void shellSort(int a[],int n)
10 {
11     int gap, i, j, temp;
12     // 初始间隔为 n/2，逐步缩小间隔
13     for (gap = n / 2; gap > 0; gap /= 2) {
14         // 对每个子序列进行插入排序
15         for (i = gap; i < n; i++) {
16             temp = a[i]; // 取出未排序序列中的元素
17             // 在子序列中从后向前扫描，找到插入位置
18             for (j = i; j >= gap && a[j - gap] > temp; j -= gap) {
19                 a[j] = a[j - gap]; // 元素后移
20             }
21             a[j] = temp; // 插入元素
22         }
23     }
24 }
25
26 int main()
27 {
28     int a[N+1],n;                /*数据存储在 a[1]...a[N] 中*/
29     printf("数据初始化...\n");
30     n=readData(a,N,"data1.txt"); /*从 data1.txt 中读入 N 个整数存入数组 a, n
   为实际读入的数据个数*/
31     printf("%d 个数据排序中...\n",n);
32     shellSort(a,n);
33     saveData(a,n,"out.txt");     /*排序结果存放在 out.txt 文件中*/
34     printf("排序结束，排序结果保存在 out.txt 文件中。\\n");
35     return 0;
36 }
```

Lab4:

```
1  /*
2  请设计简单选择排序算法函数 void selectSort(int a[],int n)，对 a[1]..a[n]进行升序
   排序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值
   时需让 N<=500000*/
7
8  /*请将本函数补充完整，并进行测试*/
9  void selectSort(int a[],int n)
10 {
11     int i, j, minIndex, temp;
12     for (i = 0; i < n - 1; i++) {
13         minIndex = i;  // 假设当前未排序部分的第一个元素是最小的
14         // 在未排序部分中找到最小元素的下标
15         for (j = i + 1; j < n; j++) {
16             if (a[j] < a[minIndex]) {
17                 minIndex = j;
18             }
19         }
20         // 将最小元素与未排序部分的第一个元素交换
21         if (minIndex != i) {
22             temp = a[i];
23             a[i] = a[minIndex];
24             a[minIndex] = temp;
25         }
26     }
27 }
28
29 int main()
30 {
31     int a[N+1],n;          /*数据存储在 a[1]...a[N] 中*/
32     printf("数据初始化...\n");
33     n=readData(a,N,"data1.txt");    /*从 data1.txt 中读入 N 个整数存入数组 a，n
   为实际读入的数据个数*/
34     printf("%d 个数据排序中...\n",n);
35     selectSort(a,n);
36     saveData(a,n,"out.txt");        /*排序结果存放在 out.txt 文件中*/
37     printf("排序结束，排序结果保存在 out.txt 文件中。\\n");
38     return 0;
39 }
```

Lab5:

```
1  /*
2  请设计筛选函数 void sift(int a[],int k,int n), 对 a[k] 进行筛选,
3  并利用其设计堆排序算法函数 void heapSort(int a[],int n),
4  对 a[1]..a[n]进行升序排序。并测试在不同数据规模下的排序效率。（详见 lab10_05.c）
5  */
6  #include "Arrayio.h"
7  #define N 10      /*N 为数据量大小, 因 data1.txt 中只有 50 万个数, 所以自行设定 N 值
8                     时需让 N<=500000*/
9
10 /*请将本函数补充完整, 并进行测试*/
11 void sift(int a[],int k,int n)
12 {
13     int i,j,finished;
14     i=k;j=2*i;
15     a[0]=a[k];
16     finished=0;
17     while((j<=n)&&(!finished))
18     {
19         if((j<n)&&(a[j+1]>a[j]))
20             j++;
21         if(a[0]>a[j])
22             finished=1;
23         else
24         {
25             a[i]=a[j];
26             i=j;j=2*j;
27         }
28     }
29     a[i]=a[0];
30 }
31
32 void heapSort(int a[],int n)
33 {
34     int i;
35     for (i=n/2;i>=1;i--)
36         sift(a,i,n);
37     for (i=n;i>1;i--)
38     {
39         a[0]=a[i];
40         a[i]=a[1];
41         a[1]=a[0];
42         sift(a,1,i-1);
43     }
```

```
43.     }
44.
45.     int main()
46.     {
47.         int a[N+1],n;                /*数据存储在 a[1]...a[N] 中*/
48.         printf("数据初始化...\n");
49.         n=readData(a,N,"data1.txt");  /*从 data1.txt 中读入 N 个整数存入数组 a, n
为实际读入的数据个数*/
50.         printf("%d 个数据排序中...\n",n);
51.         heapSort(a,n);
52.         saveData(a,n,"out.txt");      /*排序结果存放在 out.txt 文件中*/
53.         printf("排序结束，排序结果保存在 out.txt 文件中。\\n");
54.         return 0;
55.     }
```

Lab6:

```
1  /*
2  请设计冒泡排序算法函数 void bubbleSort(int a[],int n)，对 a[1]..a[n]进行升序排
   序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值
   时需让 N<=500000*/
7
8  /*请将本函数补充完整，并进行测试*/
9  void bubbleSort(int a[],int n)
10 {
11     int i, j, temp;
12     for (i = 0; i < n - 1; i++) {
13         // 标志位，用于判断是否发生交换
14         int swapped = 0;
15         // 内层循环将较大的元素“冒泡”到末尾
16         for (j = 0; j < n - 1 - i; j++) {
17             if (a[j] > a[j + 1]) {
18                 // 交换相邻元素
19                 temp = a[j];
20                 a[j] = a[j + 1];
21                 a[j + 1] = temp;
22                 swapped = 1; // 标记发生了交换
23             }
24         }
25         // 如果未发生交换，说明数组已经有序，提前退出
26         if (!swapped) {
27             break;
28         }
29     }
30 }
31
32 int main()
33 {
34     int a[N+1],n;                /*数据存储在 a[1]...a[N] 中*/
35     printf("数据初始化...\n");
36     n=readData(a,N,"data1.txt"); /*从 data1.txt 中读入 N 个整数存入数组 a, n
   为实际读入的数据个数*/
37     printf("%d 个数据排序中...\n",n);
38     bubbleSort(a,n);
39     saveData(a,n,"out.txt");     /*排序结果存放在 out.txt 文件中*/
40     printf("排序结束，排序结果保存在 out.txt 文件中.\n");
```

```
41.     return 0;  
42. }
```

Lab7:

```
1  /*
2  请设计快速排序算法函数 void quickSort(int a[],int low,int right), 对
   a[low]..a[right]进行升序排序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小, 因data1.txt 中只有50 万个数, 所以自行设定N 值
   时需让 N<=500000*/
7  /*请将本函数补充完整, 并进行测试*/
8  void quickSort(int a[],int low,int right )
9  {
10     int pi,pivot,i,j,temp;
11     if (low < right) {
12         pivot=a[right];
13         i=low - 1;
14         for (j=low; j < right; j++) {
15             if (a[j] <= pivot) {
16                 i++;
17                 temp=a[i];
18                 a[i]=a[j];
19                 a[j]=temp;
20             }
21         }
22         temp=a[i+1];
23         a[i+1]=a[right];
24         a[right]=temp;
25         pi=i+1;
26         quickSort(a, low, pi - 1);
27         quickSort(a, pi + 1, right);
28     }
29 }
30 int main()
31 {
32     int a[N+1],n;                /*数据存储在 a[1]...a[N] 中*/
33     printf("数据初始化...\n");
34     n=readData(a,N,"data1.txt"); /*从 data1.txt 中读入 N 个整数存入数组 a, n
   为实际读入的数据个数*/
35     printf("%d 个数据排序中...\n",n);
36     quickSort(a,1,n);
37     saveData(a,n,"out.txt");    /*排序结果存放在 out.txt 文件中*/
38     printf("排序结束, 排序结果保存在 out.txt 文件中.\n");
39     return 0;
40 }
```


Lab8:

```
1  /*
2  请设计归并排序算法函数 void mergeSort(int a[],int n)，对 a[1]..a[n]进行升序排
   序。
3  并测试在不同数据规模下的排序效率。
4  */
5  #include "Arrayio.h"
6  #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值
   时需让 N<=500000*/
7
8  /*请将本函数补充完整，并进行测试*/
9
10 void merge(int a[],int u,int m,int v)
11 { /*将有序段 a[u..m],a[m+1..v] 归并到 a[u..v]*/
12     int i, j, k;
13     int n1 = m - u + 1; // 左半部分的长度
14     int n2 = v - m;      // 右半部分的长度
15     int L[n1];
16     int R[n2];
17
18     // 将数据复制到临时数组
19     for (i = 0; i < n1; i++) {
20         L[i] = a[u + i];
21     }
22     for (j = 0; j < n2; j++) {
23         R[j] = a[m + 1 + j];
24     }
25
26     // 合并临时数组到原数组
27     i = 0;
28     j = 0;
29     k = u;
30     while (i < n1 && j < n2) {
31         if (L[i] <= R[j]) {
32             a[k] = L[i];
33             i++;
34         } else {
35             a[k] = R[j];
36             j++;
37         }
38         k++;
39     }
40
41     // 复制剩余元素
```

```

42.     while (i < n1) {
43.         a[k] = L[i];
44.         i++;
45.         k++;
46.     }
47.     while (j < n2) {
48.         a[k] = R[j];
49.         j++;
50.         k++;
51.     }
52. }
53.
54.
55. /*-----一趟归并-----*/
56. int mergepass(int a[],int n,int len)
57. { /*对a[1..n]进行长度为len 的一趟并归*/
58.     int i;
59.     for (i = 0; i + 2 * len - 1 < n; i += 2 * len) {
60.         merge(a, i, i + len - 1, i + 2 * len - 1);
61.     }
62.     // 处理剩余部分
63.     if (i + len - 1 < n - 1) {
64.         merge(a, i, i + len - 1, n - 1);
65.     }
66.     return len * 2; // 返回下一趟的归并长度
67.
68. }
69.
70. /*-----归并排序-----*/
71. void mergeSort(int a[],int n)
72. {
73.     int len = 1; // 初始归并长度
74.     while (len < n) {
75.         len = mergepass(a, n, len);
76.     }
77. }
78.
79. /*归并排序的递归实现*/
80. void mergeSortdc(int a[],int low,int high)
81. {
82.     if (low < high) {
83.         int mid = low + (high - low) / 2; // 计算中间位置
84.         mergeSortdc(a, low, mid); // 递归排序左半部分
85.         mergeSortdc(a, mid + 1, high); // 递归排序右半部分

```

```
86.         merge(a, low, mid, high);           // 合并两部分
87.     }
88. }
89.
90. int main()
91. {
92.     int a[N+1],n;                             /*数据存储在a[1]...a[N]中*/
93.     printf("数据初始化...\n");
94.     n=readData(a,N,"data1.txt");             /*从data1.txt 中读入N 个整数存入数组a, n
为实际读入的数据个数*/
95.     printf("%d 个数据排序中...\n",n);
96.     mergeSort(a,n);                           /* 或调用mergeSortdc(a,1,n); */
97.     saveData(a,n,"out.txt");                 /*排序结果存放在out.txt 文件中*/
98.     printf("排序结束，排序结果保存在 out.txt 文件中。 \n");
99.     return 0;
100. }
```

Lab9:

```
1  /*
2  请设计基于链表的基数排序函数 void radixSort(linklist head)，对带头结点的整型非负
   单链表进行升序排序。
3  并测试在不同数据规模下的排序效率。(注：链表中的最大整数为 500000)
4  */
5  #include "slnklist.h"
6  struct node2
7  {
8      linklist front,rear;
9  };
10 #define N 10      /*N 为数据量大小，因 data1.txt 中只有 50 万个数，所以自行设定 N 值
   时需让 N<=500000*/
11
12 /*请将本函数补充完整，并进行测试*/
13
14 void radixSort(linklist head)
15 {
16     if (head == NULL || head->next == NULL) {
17         return; // 链表为空或只有一个节点，无需排序
18     }
19
20     // 初始化队列
21     struct node2 q[10];
22     int i;
23     for (i = 0; i < 10; i++) {
24         q[i].front = q[i].rear = NULL;
25     }
26
27     // 找到链表中的最大值，确定排序的位数
28     int max = 0;
29     linklist p = head->next;
30     while (p != NULL) {
31         if (p->info > max) {
32             max = p->info;
33         }
34         p = p->next;
35     }
36     int digits = 0;
37     while (max > 0) {
38         max /= 10;
39         digits++;
40     }
41 }
```

```

42. // 按每一位进行分配和收集
43. int d;
44. for (d = 0; d < digits; d++) {
45.     // 分配过程
46.     p = head->next;
47.     while (p != NULL) {
48.         int divisor = 1;
49.         for (i = 0; i < d; i++) {
50.             divisor *= 10;
51.         }
52.         int digit = (p->info / divisor) % 10;
53.         if (q[digit].front == NULL) {
54.             q[digit].front = q[digit].rear = p;
55.         } else {
56.             q[digit].rear->next = p;
57.             q[digit].rear = p;
58.         }
59.         p = p->next;
60.     }
61.
62.     // 收集过程
63.     linklist tail = head;
64.     for (i = 0; i < 10; i++) {
65.         if (q[i].front != NULL) {
66.             tail->next = q[i].front;
67.             tail = q[i].rear;
68.             q[i].front = q[i].rear = NULL;
69.         }
70.     }
71.     tail->next = NULL; // 链表末尾置空
72. }
73. }
74. int main()
75. {
76.     linklist head;
77.     printf("数据初始化...\n");
78.     head=creatLink("data1.txt",N); /*从 data1.txt 中读入 N 个整数存入数组 a,
n 为实际读入的数据个数*/
79.     printf("数据排序中...\n");
80.     radixSort(head);
81.     writetofile(head,"out.txt"); /*排序结果保存在 out.txt 中*/
82.     delList(head);
83.     return 0;
84. }

```